

Curriculum-Driven Exploration via Adaptive Pursuit of Learnable Reward Mixtures

Group ID: 11

Abstract

Intrinsic rewards are a common approach for improving exploration in reinforcement learning, but combining them effectively with extrinsic rewards remains a challenge. In most prior work, this balance is controlled by fixed coefficients or hand-designed schedules, even though the utility of different reward signals can vary significantly across tasks and throughout training. In this work, we introduce CuReMix (Curriculum over Reward Mixtures), a method that treats the combination of intrinsic and extrinsic rewards as an automatic curriculum problem. CuReMix trains a policy conditioned on reward-mixture coefficients and adaptively prioritizes reward mixtures based on observed learning progress, while maintaining an explicit bias toward progress on the true extrinsic objective. On custom hard-exploration tasks in Crafter, CuReMix matches the performance of tuning reward-mixture coefficients separately for each task, without requiring such task-specific tuning. All code associated with this project can be found in our GitHub repository¹.

1 Introduction

The exploration-exploitation tradeoff is a defining feature of reinforcement learning (RL) and a central subject of research in the field (Sutton et al., 1998). While memoryless mechanisms for inducing exploration, such as ϵ -greedy strategies or entropy maximization, have become standard components of RL algorithms, they are often insufficient in complex environments that require deep exploration, such as sparse-reward tasks (Ladosz et al., 2022). In these settings, effective exploration often requires strategies that leverage the agent’s past interactions. A prominent way of inducing this richer form of exploration is through intrinsic reward signals whose value depends on the agent’s past experience. Intrinsic motivation has consequently

become a major theme in the RL literature, with a large body of work showing substantial gains in hard-exploration environments (Burda et al., 2018; Badia et al., 2020a; Campero et al., 2020).

The introduction of intrinsic rewards raises a key question: how should this intrinsic motivation be merged with the extrinsic environmental feedback? Existing approaches often focus on a single intrinsic reward mechanism and rely on hyperparameter tuning to determine a fixed or annealed coefficient for balancing it with the extrinsic rewards (Pathak et al., 2017; Burda et al., 2018). This rigid formulation is limiting because the usefulness of an intrinsic reward signal can naturally fluctuate over the course of an agent’s training and vary significantly across different environments or tasks (Chen et al., 2022; Yuan et al., 2023).

In this work, we propose to allow the agent to learn across the space of convex combinations of a given set of reward functions, while adaptively focusing more on those combinations that yield the greatest learning progress. By conditioning the policy on the weights assigned to each reward function, we enable the agent to explore a broader range of behaviors without committing a priori to which objectives will be most useful. This shared parameterization can allow useful behaviors discovered under one reward combination to propagate to others for which they are also beneficial. By separately estimating learning progress on each reward source, the curriculum can steer training toward reward combinations for which progress on the true extrinsic objective becomes accessible to the agent, while still providing a well-motivated mechanism for allocating training effort across intrinsic signals when extrinsic progress is limited or absent.

We term the resulting method CuReMix (Curriculum over Reward Mixtures). Instead of fixing the balance between intrinsic and extrinsic rewards in advance, CuReMix adaptively allocates train-

¹github.com/Pablo22/open-endedness-project

ing across reward mixtures based on the agent’s learning dynamics. Across our hard-exploration benchmarks, it matches the performance obtained by tuning reward-mixture coefficients separately for each evaluation task.

2 Related Work

Automatic curriculum learning methods adapt the distribution of training conditions to the agent’s evolving competence. These methods can operate over different aspects of the learning setup, such as the initial state distribution, environment parameters, or the training objective (Portelas et al., 2020). When the curriculum acts on the objective, prior work has mostly focused on sampling goals from a goal space of interest or generating goals automatically, rather than on mixtures of distinct reward signals (Sukhbaatar et al., 2017; Florensa et al., 2018; Campero et al., 2020). An exception is Fele and Babič (2025), which applies existing curriculum methods to adapt the weights assigned to a main completion signal and a set of handcrafted sparse auxiliary subgoals in simple pick-and-place and stacking tasks. In contrast, our method operates on general intrinsic reward streams that may evolve over time, making many existing curriculum methods less directly applicable.

Prior work has proposed a wide range of intrinsic reward formulations. These include curiosity-driven signals, in which the agent is encouraged to visit regions where some aspect of the environment or its interaction with it is difficult to predict or remains uncertain (Schmidhuber, 1991; Pathak et al., 2017; Burda et al., 2018), and diversity-based approaches, which incentivize the agent to collect data with broad coverage (Bellemare et al., 2016; Ostrovski et al., 2017; Hazan et al., 2019). Other lines of work cast intrinsic motivation in terms of sequences of self-generated binary goals (Sukhbaatar et al., 2017; Nair et al., 2018; Forestier et al., 2022), or in terms of skill-learning objectives aimed at acquiring behaviors that exert controllable effects on the environment (Mohamed and Jimenez Rezende, 2015; Eysenbach et al., 2018; Sharma et al., 2019).

When intrinsic rewards are used as exploration bonuses alongside an extrinsic objective, a central question is how to determine the relative influence of these rewards during training. In most work, this is handled by introducing coefficients that weight the intrinsic and extrinsic terms in a

combined objective, with these coefficients either fixed in advance through hyperparameter tuning or adjusted according to hand-designed schedules (Pathak et al., 2017; Burda et al., 2018). Some methods instead study how auxiliary rewards can be incorporated while preserving alignment with the extrinsic objective or explicitly limiting the extent to which they may degrade performance on it (Ng et al., 1999; Chen et al., 2022; Lee et al., 2024). Others avoid a single mixed objective altogether, decoupling intrinsically and extrinsically driven policies and arbitrating between exploratory and exploitative modes during training (Zhang et al., 2019; Schäfer et al., 2021; Mark et al., 2023). Another line of work uses bilevel optimization to learn an intrinsic reward function so that training under the resulting shaped objective improves extrinsic return (Zheng et al., 2018; Hu et al., 2020; Stadie et al., 2020).

Closer to our setting are methods that optimize a single policy while adaptively allocating training across candidate objectives derived from intrinsic and extrinsic rewards. AIRS does so by selecting, in real time, from a finite collection of intrinsic reward functions according to their estimated impact on extrinsic task performance (Yuan et al., 2023). NGU, by contrast, fixes a discrete set of heuristically chosen intrinsic-extrinsic weightings and trains a policy conditioned on mixtures selected uniformly from that set (Badia et al., 2020b). Agent57 refines this approach by learning separate state-action value functions for intrinsic and extrinsic signals and adaptively prioritizing which weightings to train on using a bandit driven by extrinsic return (Badia et al., 2020a). Our method differs in several respects. First, rather than restricting attention to a discrete set of candidate objectives, we consider a continuous space of reward mixtures. Second, the policy is conditioned directly on the reward weights themselves, allowing it to exploit the structure of this space rather than leaving the objective implicit as in AIRS or indexing each objective by a separate one-hot label as in NGU and Agent57. Third, objective selection is driven by learning progress rather than return, so that objectives yielding high return but little further improvement are naturally deprioritized. Moreover, by incorporating progress on intrinsic reward streams, our method can still prioritize promising objectives even when the extrinsic reward provides little or no guidance.

3 Method

We introduce CuReMix, an automatic curriculum learning framework that adaptively leverages intrinsic rewards to facilitate progress on the extrinsic task.

We consider a Markov Decision Process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, and transition dynamics $P(s'|s, a)$. Although we adopt the MDP framework for notational simplicity, the method extends directly to partially observable settings (Section 4). We assume access to $K \geq 2$ reward functions ρ_k with corresponding discount factors γ_k for $k \in \{0, \dots, K-1\}$. Here, ρ_0 is the extrinsic reward function that is part of the environment, and $\rho_1, \dots, \rho_{K-1}$ are intrinsic reward functions designed to drive exploration.

To train the policy, feedback from these reward functions must be distilled into a single scalar signal. Rather than fixing this combination in advance or specifying a hand-designed schedule, CuReMix adapts it online based on the agent’s learning dynamics. We focus on the space of convex combinations of the reward functions. Let α_k denote the weight assigned to ρ_k . Then the weight vector $\alpha = (\alpha_0, \dots, \alpha_{K-1}) \in \mathbb{R}^K$ satisfies $\mathbf{1}^\top \alpha = 1$ and $\alpha_k \geq 0 \quad \forall k$. Each valid α can be viewed as specifying a composite reward function $\rho_\alpha = \sum_{k=0}^{K-1} \alpha_k \rho_k$. This induces a continuous space of candidate training objectives.

Although this description is conceptually convenient, directly using these raw reward mixtures would make the relative trade-offs encoded by α poorly calibrated due to scale differences and non-stationarity in the reward streams. CuReMix instead considers a α -parameterized objective space obtained by weighting calibrated per-stream signals, as described in Section 3.2.

The central idea of CuReMix is to learn an α -conditioned policy, $\pi(a|s, \alpha)$, and to apply an automatic curriculum over this constructed continuous task space to adaptively select which reward mixtures the agent trains on.

3.1 Training Curriculum

In multi-task settings, learning progress (LP) measures the agent’s rate of improvement on a task. Prioritizing tasks with high learning progress allows the agent to focus training on regions of the task space with valuable feedback, while avoiding effort on tasks that are already trivial, currently out of reach, or dominated by uncontrollable variability.

Learning-progress-driven curricula can substantially accelerate learning in goal-conditioned RL (Colas et al., 2019; Kanitscheider et al., 2021).

In standard goal-conditioned RL, different goals are typically treated as equally important. In our setting, however, the problem is fundamentally asymmetric: the ultimate objective corresponds uniquely to the extrinsic task defined by $\alpha = (1, 0, \dots, 0)$. To account for this asymmetry, we propose a task-desirability score to guide the curriculum that combines learning progress on the task’s α -weighted objective with progress on the extrinsic return.

Let $\tilde{LP}_k(\alpha)$ denote the normalized learning progress of the α -conditioned policy on reward function ρ_k . We define the task-desirability score for a given weight vector α as:

$$\text{Score}(\alpha) = (1 - \lambda) LP_{\text{tot}}(\alpha) + \lambda LP_{\text{ext}}(\alpha) \quad (1)$$

where $LP_{\text{tot}}(\alpha) = \sum_{k=0}^{K-1} \alpha_k \tilde{LP}_k(\alpha)$ and $LP_{\text{ext}}(\alpha) = LP_0(\alpha)$.

The LP_{ext} term, which measures progress on the extrinsic reward stream, biases the curriculum toward reward mixtures in which the agent is making progress on the true task of interest. The LP_{tot} term, in contrast, captures improvement on the composite training objective induced by α , and provides a justified signal for adaptively allocating training among exploration objectives even in the absence of extrinsic feedback.

To implement this curriculum, CuReMix trains a score-prediction network $f(\alpha)$ in an online fashion. At each curriculum iteration, we sample a batch of α vectors. After training the policy on these mixtures, we obtain empirical estimates of their respective scores. The network f is then updated using empirical score estimates from the most recent iterations. To select the next batch of reward mixtures, we treat $f(\alpha)$ as an unnormalized density function over the space of valid α and sample from it. Through this iterative process, CuReMix induces an adaptive curriculum that concentrates training on reward combinations that yield informative learning signals.

3.2 Training Objectives

Reward functions can operate on substantially different return scales. Moreover, many intrinsic reward functions are inherently non-stationary due to their dependence on adaptive components. As a result, the relative contribution of each reward

stream to both learning signals and curriculum decisions can become poorly calibrated and drift over time. This is particularly problematic in our setting, where the coefficients α are intended to parameterize meaningful trade-offs across reward functions. To ensure that these trade-offs remain consistent throughout training and are not dominated by scale differences, it is necessary to introduce normalization mechanisms that preserve the intended comparability across streams.

3.2.1 Score Estimation

Beyond scale differences, non-stationarity in intrinsic reward functions introduces an additional challenge for learning progress estimation. Since these rewards evolve over time, changes in measured performance may reflect modifications of the reward function itself rather than genuine improvements in the policy. To address this, we estimate learning progress using only data collected over intervals during which all intrinsic reward functions are held fixed.

Within these intervals, we estimate the learning progress LP_k for each reward function k independently and normalize it using a running estimate of the standard deviation of its returns to ensure comparability across reward streams. This procedure yields the normalized learning progress estimates $\tilde{LP}_k$, which are used to compute the task-desirability score (Eq. 1). The score predictor $f(\alpha)$ is trained by minimizing the squared error between its predictions and these empirical score targets:

$$\mathcal{L}_f = \frac{1}{|B|} \sum_B \left(f(\alpha) - \widehat{\text{Score}}(\alpha) \right)^2 \quad (2)$$

where B is a replay buffer of recently obtained $(\alpha, \widehat{\text{Score}}(\alpha))$ pairs. A detailed description of the LP_k estimation and normalization procedures is provided in Appendix A.1 and A.2.2.

3.2.2 Agent Learning

We train the agent with Proximal Policy Optimization (PPO) (Schulman et al., 2017). We maintain separate value heads $V_k(s; \alpha)$ for each of the K reward streams and normalize each reward stream using a running estimate of the standard deviation of its returns to promote stable value learning. Although value heads are separate, they remain conditioned on α , since the policy’s behavior and therefore the induced trajectory distribution depend on

the target reward mixture specified by α . We define the value loss as:

$$L_V(s; \alpha) = \frac{1}{2} \sum_{k=0}^{K-1} \alpha_k \left(V_k(s; \alpha) - \hat{G}_{k,s} \right)^2 \quad (3)$$

where $\hat{G}_{k,s}$ is the empirical return-to-go from state s computed from the normalized rewards of stream k .

Advantages $\hat{A}_k(s, a; \alpha)$ are estimated independently for each reward stream using Generalized Advantage Estimation and then standardized per stream to mitigate scale discrepancies. Let $\tilde{A}_k(s, a; \alpha)$ denote the resulting standardized advantage estimate for reward stream k obtained from data collected by the policy when conditioned on α . We then construct the final advantage used in the PPO policy update as:

$$\hat{A}(s, a; \alpha) = \sum_{k=0}^{K-1} \alpha_k \tilde{A}_k(s, a; \alpha). \quad (4)$$

A high-level overview of CuReMix is provided in Algorithm 1, while additional implementation details are given in the appendix.

4 Experiments

4.1 Environment and Tasks

We evaluated CuReMix on custom sparse-reward tasks constructed in Craftax-Classic (Matthews et al., 2024), the JAX implementation of Crafter (Hafner, 2021). Crafter is a challenging exploration benchmark in which agents must navigate a diverse 2D world to complete achievements through behaviors such as resource collection, crafting, and combat. For computational efficiency, we used symbolic observations.

We modified the default environment in two ways. First, instead of rewarding the full set of achievements, we construct sparse-reward tasks by assigning extrinsic reward to only one or two target achievements, making intrinsic motivation more important for effective exploration. Second, we disabled procedural generation of new environment layouts in between episodes to make the benchmark more compatible with our specific choice of intrinsic reward functions. These design choices were fixed before evaluating CuReMix.

In particular, we considered four tasks spanning different dependency structures among achieve-

Algorithm 1: CuReMix

```

1 Initialize:  $\alpha$ -conditioned policy  $\pi$ , intrinsic reward
  functions  $\{\rho_k\}_{k=1}^{K-1}$ ,  $\alpha$ -conditioned critics
   $\{V_k\}_{k=0}^{K-1}$ , score predictor  $f$ , and score buffer  $B$ 
2 while training do
3   Sample a batch of environments  $\{M_i\}_{i=1}^N$ 
    $\triangleright$  Sample new batch of  $\alpha$  values
4   Sample  $\{\alpha^{(i)}\}_{i=1}^N$  by treating  $f(\alpha)$  as an
   unnormalized density over valid  $\alpha$ 
    $\triangleright$  Collect data and train agent
5    $D_{i\_rewards\_update} \leftarrow \emptyset$ 
6    $D_{score\_estimation} \leftarrow \emptyset$ 
7   for  $\tau = 1$  to  $num\_updates\_per\_env$  do
8      $D_{agent\_update} \leftarrow \emptyset$ 
9     foreach  $M_i$  do
10      Collect data with  $\pi(\cdot|s, \alpha^{(i)})$  on  $M_i$ 
11      Compute intrinsic rewards over the
      collected timesteps using all intrinsic
      reward functions  $\{\rho_k\}_{k=1}^{K-1}$ 
12      Append transitions and required
      metadata to  $D_{agent\_update}$ ,
       $D_{i\_rewards\_update}$ , and  $D_{score\_estimation}$ 
13    end
14    Update  $\pi$  and  $\{V_k\}_{k=0}^{K-1}$  on  $D_{agent\_update}$ 
    according to Sec. 3.2.2
15  end
   $\triangleright$  Update intrinsic reward
  functions
16  Update  $\{\rho_k\}_{k=1}^{K-1}$  on  $D_{i\_rewards\_update}$ 
   $\triangleright$  Estimate scores and train
  score predictor
17  Estimate  $\widehat{LP}_k(\alpha^{(i)})$  for all  $k \in \{0, \dots, K-1\}$ 
  and  $i \in \{1, \dots, N\}$  from  $D_{score\_estimation}$ 
  according to App. A.1 and A.2.2
18  Compute  $\widehat{Score}(\alpha^{(i)})$  for each  $i \in \{1, \dots, N\}$ 
  using Eq. 1
19  Push  $\{(\alpha^{(i)}, \widehat{Score}(\alpha^{(i)}))\}_{i=1}^N$  into buffer  $B$ 
  (replace the  $N$  oldest tuples if full)
20  Update score predictor  $f(\alpha)$  on data from  $B$  to
  minimize  $\mathcal{L}_f$  (Equation 2)
21 end

```

ments. Two tasks reward a single target achievement: *collect iron* and *make stone sword*. The remaining two reward pairs of achievements: *defeat skeleton & make stone pickaxe*, which combines two largely unrelated achievements, and *place furnace & make iron pickaxe*, in which one achievement is a prerequisite for the other. Table 1 summarizes the rewarded achievements and total training budget for each task.

4.2 Baselines and Experimental Setup

We compared CuReMix against two baselines: an extrinsic-only agent trained without intrinsic rewards, and a family of fixed-weights baselines trained with constant reward-mixture coefficients throughout training. The extrinsic-only baseline isolates the effect of adding intrinsic rewards, while

Task	Timesteps
collect iron	500M
make stone sword	500M
defeat skeleton & make stone pickaxe	500M
place furnace & make iron pickaxe	1B

Table 1: Achievements rewarded and total computational budget for each task.

the fixed-weight baselines reflect task-specific tuning of reward coefficients. For the latter, we sample coefficients from a grid with a step size of 0.125, subject to the constraints that the weights sum to 1 and that at least 0.125 of the total weight is assigned to the extrinsic reward.

Both CuReMix and the fixed-weights baselines use the same two intrinsic reward mechanisms: Intrinsic Curiosity Module (ICM) (Pathak et al., 2017) and Random Network Distillation (RND) (Burda et al., 2018). To make the two signals more complementary, we restricted RND to the agent’s inventory features, so that it rewards novel changes in inventory state rather than generic movement through the environment.

All methods share the same optimization setup, hyperparameters, and base Gated Transformer-XL agent architecture (Dai et al., 2019; Parisotto et al., 2020). No hyperparameter tuning was performed for CuReMix-specific components. Further information is provided in the Appendix.

5 Results and Discussion

5.1 Performance and Sample Efficiency

Figure 1 compares final performance across all four tasks for CuReMix and the baselines. In these sparse-reward settings, the extrinsic-only baseline fails to make meaningful progress, confirming that optimizing the target reward alone is insufficient and that effective exploration is essential.

Among fixed-weight baselines, performance depends strongly on the choice of reward-mixture coefficients, and the best coefficients are not generally the best across all tasks. This variability highlights a central limitation of fixed reward mixtures: good coefficients require task-specific tuning and therefore cannot be generally amortized across evaluation tasks. Despite not performing such tuning, CuReMix essentially matches the final performance of the best fixed-weight baseline on all four tasks.

A notable feature of CuReMix is that the final

learned policy often achieves strong extrinsic performance when conditioned on a broad range of reward mixtures, including mixtures for which training a policy from scratch with those same fixed weights performs poorly. We believe the most likely reason is that behaviors acquired under one reward combination can transfer to other reward combinations within the shared conditioned policy. In particular, on two tasks, conditioning the final CuReMix policy on the purely extrinsic objective, $\alpha = (1, 0, 0)$, roughly yields maximum performance. If transfer across conditionings were fully effective, one might expect this to hold more generally, since $\alpha = (1, 0, 0)$ corresponds to the most directly exploitative setting. Figure 3 shows that this conditioning does attain the best performance at intermediate stages of training, but in some seeds it does not retain that advantage by the end. One possible explanation is that mixtures emphasizing extrinsic rewards are deprioritized early because they initially show little learning progress and are not revisited often enough later in training.

Figure 2 shows the performance of the methods throughout training. Relative to the single best fixed-weight baseline, chosen in hindsight for each task, CuReMix converges more slowly. However, this comparison assumes oracle access to the best task-specific coefficients, whereas CuReMix remains competitive with the strongest values without requiring any such search.

5.2 Curriculum Adaptation

To understand CuReMix’s behavior, we analyzed how the reward weights evolve during training. As depicted in Figure 3, all tasks showed similar patterns. At the beginning of the training, ICM’s weight increased. This was followed by a later decrease in ICM’s weight in favor of RND. This makes sense because, during this phase, the agent barely knows how to add objects to the inventory and does not receive any extrinsic reward. As the agent begins to acquire crafting and mining skills, the RND coefficient increases, driving exploration that eventually yields extrinsic rewards. Upon receiving extrinsic feedback, the curriculum shifts weight toward the extrinsic reward, before ultimately decreasing it again once learning progress on that objective stabilizes. This capacity to adapt the mixture of coefficients suggests that the score-predictor network is successfully influencing the weightings by favoring those with higher learning

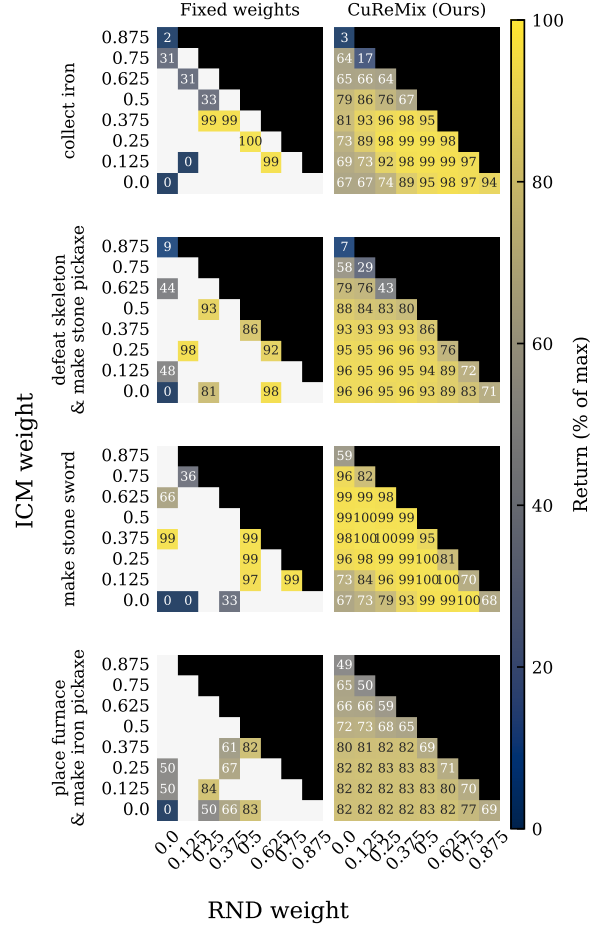


Figure 1: Performance across all tasks after training with fixed coefficients compared with the final policy of CuReMix conditioned on different weightings. Cell color encodes the mean final return across three seeds, expressed as a percentage of the maximum. Black cells mark invalid ICM/RND weight combinations in which the two weights sum to more than 0.875, thereby guaranteeing at least 0.125 to the extrinsic reward. White cells mark cells with no data.

progress.

Furthermore, Figure 4 shows the relationship between the distribution of α sampled during training and their associated performance for the task *collect iron*. In addition to observing the same evolution of the reward mixture as already described, we observe how the sampling of these mixtures focuses mainly on regions where a higher expected extrinsic return is achieved. Interestingly, the best region identified at the end of training matches the one found by the random search, suggesting that CuReMix can dynamically identify effective reward mixtures. Please refer to Appendix B for additional figures, including those related to other tasks.

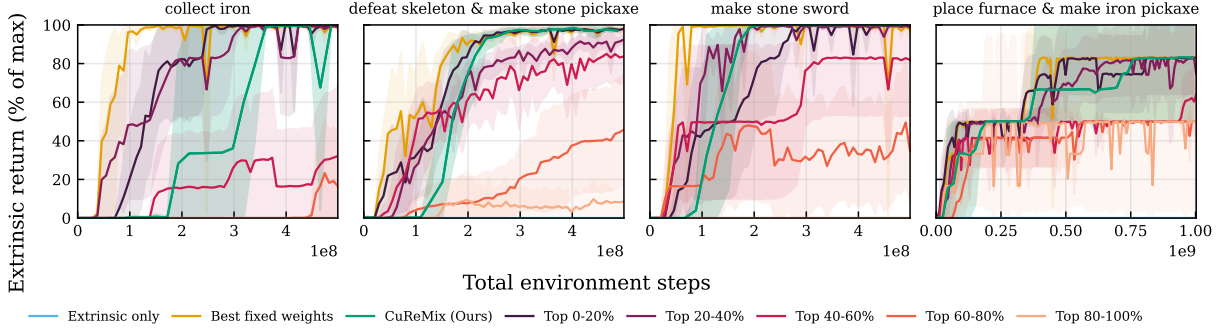


Figure 2: Mean return expressed as a percentage of the maximum as training progresses in each of the four evaluation tasks. Each curve is the mean extrinsic return across three seeds. Shaded bands show the min-max range across seeds except for the fixed-weight baselines grouped by final-performance quintile (Top 0-20% to Top 80-100%), where they show a two-sided 95% confidence interval. The best fixed weights for each task were selected by first restricting to candidates with final performance within 0.02 of the best final return, and then choosing the one with the greatest area under the learning curve.

6 Conclusion

Traditionally, intrinsic motivation in sparse-reward environments relies on static combinations of reward signals, which assumes their relative importance is constant throughout training and across tasks. CuReMix, however, offers a more flexible alternative by functioning as a dynamic curriculum generator.

When compared with the top values of α from a random search over fixed coefficients, our approach matched their final performance, despite zero hyperparameter tuning. Moreover, conditioning CuReMix on these best-performing α values during evaluation consistently yields high-performing results. While CuReMix converges more slowly than the best-performing α , our method is still significantly faster than a random coefficient sweep and roughly matches the convergence speed of the top 40% of fixed reward mixtures. These findings suggest that CuReMix can provide a robust alternative to fixed-coefficient search methods that generalizes across tasks without exhaustive tuning.

6.1 Limitations and Future Work

This study has several limitations.

Most notably, the scope of our evaluation is limited. We only measured performance on two fixed intrinsic reward functions. As such, we did not assess how CuReMix scales with additional intrinsic rewards, given that the search space over coefficients scales exponentially with the number of reward functions. Moreover, this means that we do not test CuReMix’s ability to de-prioritize (deliberately) noisy intrinsic rewards. Additionally, we

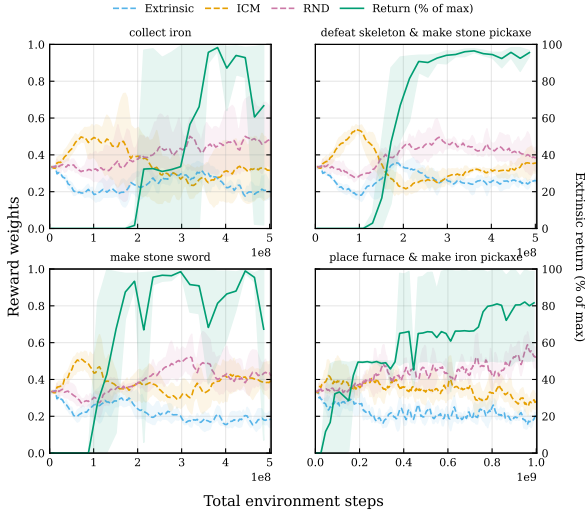


Figure 3: Curriculum adaptation and performance for each task. Dashed curves (left y-axis) show the mean curriculum weights assigned to extrinsic, ICM, and RND rewards across three seeds, with shaded bands indicating min-max variation across seeds. The solid green curve (right y-axis) shows mean extrinsic return for CuReMix’s policy conditioned with 100% of the weight to the extrinsic reward (i.e., $\alpha = (1, 0, 0)$), normalized as a percentage of the task maximum, with min-max shading across seeds.

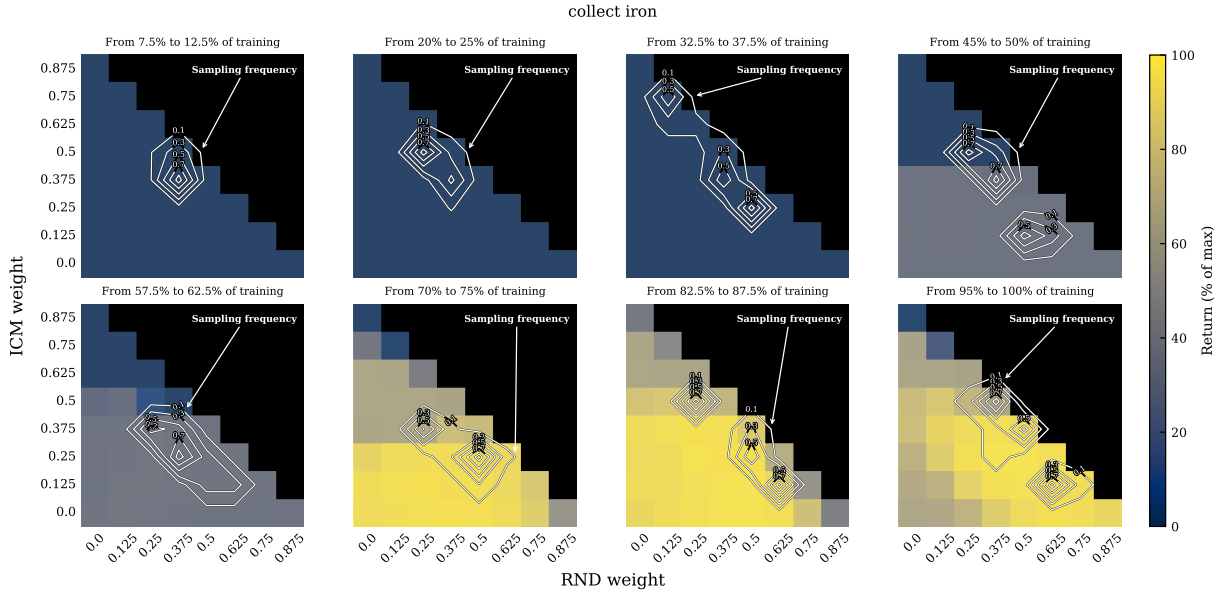


Figure 4: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *collect iron* task, shown at eight intervals comprising 5% of training each. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay the normalized curriculum sampling frequency in weight space over their respective intervals, with labels indicating the contour levels (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

measured results only for four achievement combinations within the Crafter environment. A more comprehensive study evaluating CuReMix across a variety of tasks and environments is required to assess whether our conclusions generalize. Furthermore, the fact that non-adaptive methods like simple fixed coefficients were able to achieve full convergence indicates that train-time adaptation may not be needed for optimal learning in sparse tasks in Crafter. Future work should evaluate CuReMix in a variety of complex environments with intrinsic rewards varying in number and type.

Additionally, we did not perform hyperparameter tuning for CuReMix or the intrinsic reward functions, so it is unclear how sensitive our approach is to improper hyperparameters. In a similar vein, we did not conduct ablations on any design decisions, so it is unclear which components of CuReMix contribute most to our observed results. Future work could focus on investigating these two aspects of CuReMix.

Lastly, we compared our method only to the baseline of random search over the coefficients. To thoroughly assess our contribution to the body of work, future work should directly compare CuReMix with similar methods, such as Agent57 (Badia et al., 2020a) and AIRS (Yuan et al., 2023).

References

- Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Zhaoan Daniel Guo, and Charles Blundell. 2020a. Agent57: Outperforming the atari human benchmark. In *International conference on machine learning*, pages 507–517. PMLR.
- Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Bilal Piot, Steven Kapturowski, Olivier Tieleman, Martín Arjovsky, Alexander Pritzel, Andrew Bolt, et al. 2020b. Never give up: Learning directed exploration strategies. *arXiv preprint arXiv:2002.06038*.
- Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. 2016. Unifying count-based exploration and intrinsic motivation. *Advances in neural information processing systems*, 29.
- Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2018. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*.
- Andres Campero, Roberta Raileanu, Heinrich Küttler, Joshua B Tenenbaum, Tim Rocktäschel, and Edward Grefenstette. 2020. Learning with amigo: Adversarially motivated intrinsic goals. *arXiv preprint arXiv:2006.12122*.
- Eric Chen, Zhang-Wei Hong, Joni Pajarinen, and Pulkit Agrawal. 2022. Redeeming intrinsic rewards via constrained optimization. *Advances in neural information processing systems*, 35:4996–5008.

- Cédric Colas, Pierre Fournier, Mohamed Chetouani, Olivier Sigaud, and Pierre-Yves Oudeyer. 2019. Curious: intrinsically motivated modular multi-goal reinforcement learning. In *International conference on machine learning*, pages 1331–1340. PMLR.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*.
- Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 2018. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*.
- Benjamin Fele and Jan Babič. 2025. Curriculum learning algorithms for reward weighting in sparse reward robotic manipulation tasks. *IEEE Access*, 13:45544–45558.
- Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. 2018. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pages 1515–1528. PMLR.
- Sébastien Forestier, Rémy Portelas, Yoan Mollard, and Pierre-Yves Oudeyer. 2022. Intrinsically motivated goal exploration processes with automatic curriculum learning. *Journal of Machine Learning Research*, 23(152):1–41.
- Danijar Hafner. 2021. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*.
- Elad Hazan, Sham Kakade, Karan Singh, and Abby Van Soest. 2019. Provably efficient maximum entropy exploration. In *International Conference on Machine Learning*, pages 2681–2691. PMLR.
- Yujing Hu, Weixun Wang, Hangtian Jia, Yixiang Wang, Yingfeng Chen, Jianye Hao, Feng Wu, and Changjie Fan. 2020. Learning to utilize shaping rewards: A new approach of reward shaping. *Advances in Neural Information Processing Systems*, 33:15931–15941.
- Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebggen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, et al. 2021. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*.
- Pawel Ladosz, Lilian Weng, Minwoo Kim, and Hyondong Oh. 2022. Exploration in deep reinforcement learning: A survey. *Information Fusion*, 85:1–22.
- Chi-Chang Lee, Zhang-Wei Hong, and Pulkit Agrawal. 2024. Going beyond heuristics by imposing policy improvement as a constraint. *Advances in Neural Information Processing Systems*, 37:138032–138087.
- Max Sobol Mark, Archit Sharma, Fahim Tajwar, Rafael Rafailov, Sergey Levine, and Chelsea Finn. 2023. Offline retraining for online rl: Decoupled policy learning to mitigate exploration bias. *arXiv preprint arXiv:2310.08558*.
- Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Thomas Jackson, Samuel Coward, and Jakob Nicolaus Foerster. 2024. *Craftax: A lightning-fast benchmark for open-ended reinforcement learning*. In *Forty-first International Conference on Machine Learning*.
- Shakir Mohamed and Danilo Jimenez Rezende. 2015. Variational information maximisation for intrinsically motivated reinforcement learning. *Advances in neural information processing systems*, 28.
- Ashvin V Nair, Vitchyr Pong, Murtaza Dalal, Shikhar Bahl, Steven Lin, and Sergey Levine. 2018. Visual reinforcement learning with imagined goals. *Advances in neural information processing systems*, 31.
- Andrew Y Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287. Citeseer.
- Georg Ostrovski, Marc G Bellemare, Aäron Oord, and Rémi Munos. 2017. Count-based exploration with neural density models. In *International conference on machine learning*, pages 2721–2730. PMLR.
- Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. 2020. Stabilizing transformers for reinforcement learning. In *International conference on machine learning*, pages 7487–7498. PMLR.
- Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. 2017. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR.
- Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. 2020. Automatic curriculum learning for deep rl: A short survey. *arXiv preprint arXiv:2003.04664*.
- Lukas Schäfer, Filippos Christianos, Josiah P Hanna, and Stefano V Albrecht. 2021. Decoupled reinforcement learning to stabilise intrinsically-motivated exploration. *arXiv preprint arXiv:2107.08966*.
- Jürgen Schmidhuber. 1991. Curious model-building control systems. In *Proc. international joint conference on neural networks*, pages 1458–1463.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. *Proximal policy optimization algorithms*.
- Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. 2019. Dynamics-aware unsupervised discovery of skills. *arXiv preprint arXiv:1907.01657*.

- Bradly Stadie, Lunjun Zhang, and Jimmy Ba. 2020. Learning intrinsic rewards as a bi-level optimization problem. In *Conference on uncertainty in artificial intelligence*, pages 111–120. PMLR.
- Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. 2017. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*.
- Richard S Sutton, Andrew G Barto, et al. 1998. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge.
- Mingqi Yuan, Bo Li, Xin Jin, and Wenjun Zeng. 2023. Automatic intrinsic reward shaping for exploration in deep reinforcement learning. In *International Conference on Machine Learning*, pages 40531–40554. PMLR.
- Jingwei Zhang, Niklas Wetzal, Nicolai Dorka, Joschka Boedecker, and Wolfram Burgard. 2019. Scheduled intrinsic drive: A hierarchical take on intrinsically motivated exploration. *arXiv preprint arXiv:1903.07400*.
- Zeyu Zheng, Junhyuk Oh, and Satinder Singh. 2018. On learning intrinsic rewards for policy gradient methods. *Advances in neural information processing systems*, 31.

A Appendix

A.1 Learning Progress Estimation

Intrinsic reward functions present a challenge in estimating learning progress across long time-spans because many of them are non-stationary. If performance were measured at two distant points in time without accounting for this, changes in the reward function itself would confound the measurement of the policy’s improvement. To avoid this issue, we estimate learning progress only from data collected over intervals during which all intrinsic reward functions are kept fixed.

In CuReMix, each curriculum iteration performs M consecutive on-policy training iterations, each comprising data collection followed by PPO optimization, before updating the intrinsic reward functions. For each sampled α and reward function ρ_k , we estimate the raw learning progress $LP_k(\alpha)$ as the slope of a linear fit over performance measurements collected across these M batches. The construction of the regression dataset (x, y) depends on the nature of the reward stream.

A.1.1 Episodic Reward Functions

For episodic reward functions, each completed episode initially defines one candidate data point.

Y-Coordinate: Performance is measured using the undiscounted full-episode return under reward function ρ_k .

X-Coordinate: An episode may span multiple policy-update batches. Let p_m be the fraction of the episode’s total steps that were executed during policy update batch $m \in \{1, \dots, M\}$. The x -coordinate for the episode is its step-weighted batch index:

$$x = \sum_{m=1}^M p_m \cdot m$$

Aggregation and Weights: Episodes that start and terminate within the same batch m are aggregated into a single data point. Its y -value is the mean return of those episodes, and the data point is assigned weight $w = \sqrt{N}$, where N is the number of aggregated episodes. Episodes spanning multiple batches are not aggregated and remain individual data points with weight $w = 1$.

The raw learning progress $LP_k(\alpha)$ is estimated as the slope of a weighted least squares fit over the resulting dataset.

A.1.2 Non-Episodic Reward Functions

For non-episodic reward functions, each batch m contributes one data point with x -coordinate $x_m = m$. The performance value y_m , is an approximation of the expected infinite-horizon return for the policy. Let $\bar{r}_{m,k}$ denote the mean reward from ρ_k across all transitions collected in batch m . We measure performance as $y_m = \frac{\bar{r}_{m,k}}{1-\gamma_k}$.

The raw learning progress $LP_k(\alpha)$ is estimated as the slope of an ordinary least squares fit over the M resulting points.

A.2 Normalization

This appendix details how we estimate and maintain normalization statistics for: (i) stabilizing value-function learning, and (ii) making learning-progress estimates comparable across reward functions.

A.2.1 Reward Normalization for Policy Learning

For policy learning, we normalize the raw rewards from each reward function ρ_k using an online estimate of the standard deviation of a running empirical proxy return. For episodic streams, we reset the return accumulation at episode boundaries: $R_{t,k} = r_{t,k} + \gamma_k(1 - d_{t-1})R_{t-1,k}$. For non-episodic streams, episode boundaries are ignored:

$R_{t,k} = r_{t,k} + \gamma_k R_{t-1,k}$. We maintain an online estimate of the standard deviation of $R_{t,k}$, denoted σ_k^{train} , and normalize rewards as:

$$\tilde{r}_{t,k} = \frac{r_{t,k}}{\sigma_k^{\text{train}} + \varepsilon}.$$

These normalized rewards are used to compute the value targets and per-stream GAE estimates.

A.2.2 Learning Progress Normalization for Score Estimation

For each reward function ρ_k , we maintain an estimate of its respective return’s variance $(\sigma_k^{LP})^2$. This variance is maintained as an exponentially moving average that is updated once per curriculum iteration. Let $\sigma_{\text{iter},k}^2$ be the variance computed from all trajectories collected during the current iteration. The EMA is updated via: $(\sigma_k^{LP})^2 \leftarrow (1 - \beta)(\sigma_k^{LP})^2 + \beta\sigma_{\text{iter},k}^2$. The computation of the iteration’s return variance $\sigma_{\text{iter},k}^2$ depends on the nature of the reward stream.

For episodic reward streams, we estimate the variance using the observed undiscounted full-episode returns, which are also the measure of performance used for LP estimation (Appendix A.1). To prevent environments where the agent terminates rapidly from artificially dominating the variance estimate, we compute an environment-balanced iteration variance. Specifically, each completed episode contributes to $\sigma_{\text{iter},k}^2$ with a weight inversely proportional to the total number of completed episodes in its respective environment during that iteration.

For non-episodic reward streams, we estimate $\sigma_{\text{iter},k}^2$ as the variance of a running empirical proxy return that spans all the data collected in that iteration. We initialize the accumulator as $R_{0,k} = \frac{\bar{r}_k}{1-\gamma_k}$, where $\bar{r}_k$ is the mean raw reward for the environment on that iteration. Each environment maintains its own accumulator, and the proxy returns across all environments are pooled to compute the iteration variance $\sigma_{\text{iter},k}^2$.

Raw learning progress estimates (Appendix A.1) are normalized by this standard deviation before being aggregated into the final task-desirability score:

$$\tilde{LP}_k(\alpha) = \frac{LP_k(\alpha)}{\sigma_k^{LP} + \varepsilon}. \quad (5)$$

A.3 Implementation Details and Hyperparameters

A.3.1 Data Collection and Policy Optimization

We collect experience in parallel from batches of 1024 environments and collect 4096 transitions per environment. We perform updates with PPO every 256 steps are collected in every environment. For PPO we use a learning rate of 2×10^{-4} , clipping ratio of 0.2, entropy coefficient of 0.01, and value function coefficient of 0.5. For the extrinsic reward stream we use $\gamma = 0.99$ and GAE $\lambda = 0.95$. We optimize policies using the Adam optimizer.

A.3.2 Policy Architecture

Observations from the Craftax-Classic symbolic environment (map, inventory, and status channels) are processed by a structured encoder that embeds each channel separately before projecting to a shared 256-dimensional representation. These are concatenated and passed through a single-block Gated Transformer-XL with hidden size 192, 4 attention heads (head dimension 48), and gating bias 2.0 (Parisotto et al., 2020). Separate actor and critic heads are 2-layer MLPs with a hidden dimension of 256. The critic produces K output values, one per reward stream, enabling per-stream value estimation under the current α . GELU and ReLU activations are used throughout. In CuReMix (curriculum) mode, the α vector is concatenated to the transformer input, the actor-head input, and the critic-head input, giving the policy explicit awareness of the current reward mixture. For baselines α -conditioning is disabled.

A.3.3 ICM

The Intrinsic Curiosity module (Pathak et al., 2017) maintains a shared observation encoder (same architecture as the one used by the policy) alongside two separate MLPs: a forward model and an inverse model each with two hidden layers of size 256 and ReLU activations. The forward model predicts the next state embedding $\hat{z}_{t+1}$ from the current embedding z_t and the one-hot action a_t . The inverse model predicts the action logits from the concatenation of z_t and z_{t+1} , trained with the softmax cross-entropy. The combined loss term uses $\beta = 0.2$. The forward loss uses a stop-gradient on z_{t+1} so that only the forward model is updated through it. The module is trained with Adam using 1 epoch and 64 mini-batches per update. The ICM value stream uses $\gamma = 0.99$ and GAE $\lambda = 0.95$.

A.3.4 RND

Random Network Distillation (Burda et al., 2018) maintains two networks with identical architecture: a fixed random target and a trained predictor. Both use a three-layer MLP with hidden dimensions (256, 128, 128), taking only the features corresponding to the agent’s inventory as input, and projecting to an output embedding of dimension 128. The predictor is trained to match the target’s embedding of the next observation. The predictor is updated with Adam using $\text{lr} = 10^{-4}$, $\varepsilon = 10^{-5}$, 1 epoch and 64 mini-batches per update. The RND reward stream uses $\gamma = 0.995$ and GAE $\lambda = 0.95$.

A.3.5 CuReMix curriculum

The score function weights extrinsic and total learning progress equally $\lambda = 0.5$. The replay buffer retains empirical score estimates from the 3 most recent batches. The score predictor is a three-layer MLP with hidden dimension 64 and ReLU activations. We use Adam with $\text{lr}=10^{-4}$, 1 epoch, and 64 mini-batches per update for optimization. New α values are drawn with importance sampling, and predictor-guided sampling begins after 3 warm-up uniform batches.

B Other Figures

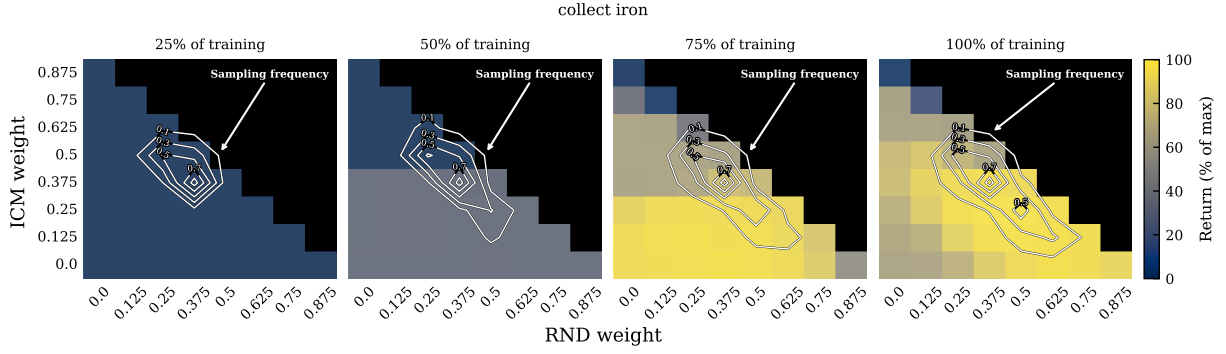


Figure 5: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *collect iron* task, shown at 25%, 50%, 75%, and 100% of training. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay normalized curriculum sampling frequency in weight space up to that stage, with labels indicating contour level (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

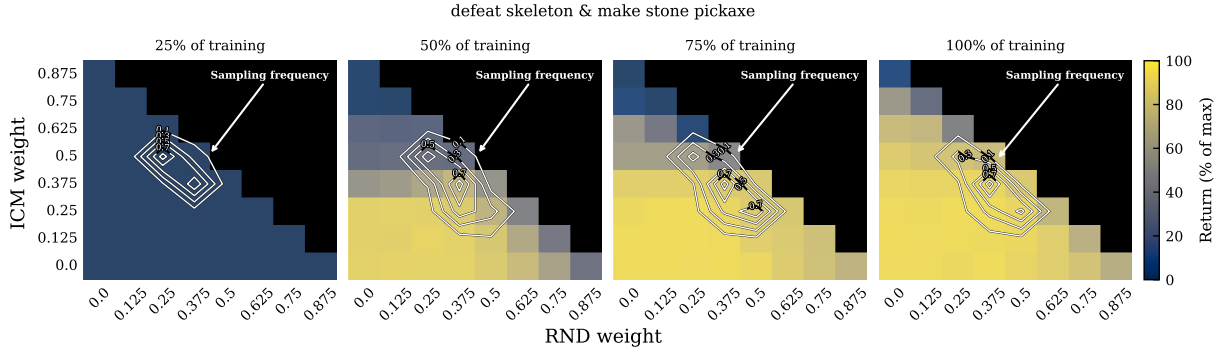


Figure 6: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *defeat skeleton & make stone pickaxe* task, shown at 25%, 50%, 75%, and 100% of training. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay normalized curriculum sampling frequency in weight space up to that stage, with labels indicating contour level (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

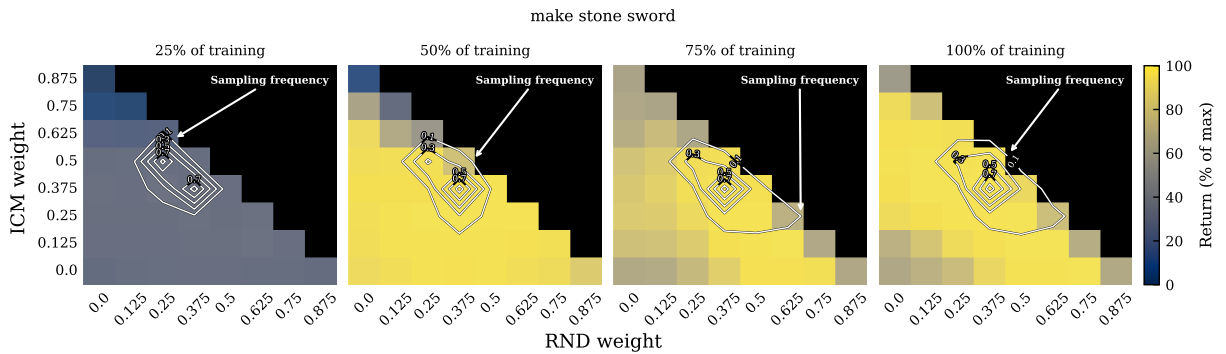


Figure 7: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *make stone sword* task, shown at 25%, 50%, 75%, and 100% of training. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay normalized curriculum sampling frequency in weight space up to that stage, with labels indicating contour level (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

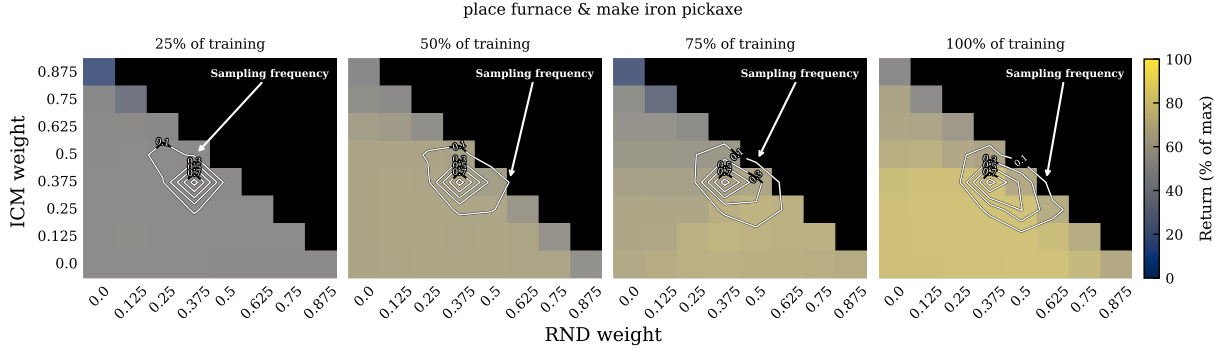


Figure 8: Training-stage snapshots of the CuReMix-conditioned policy landscape for the “place furnace & make iron pickaxe” task, shown at 25%, 50%, 75%, and 100% of training. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay normalized curriculum sampling frequency in weight space up to that stage, with labels indicating contour level (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

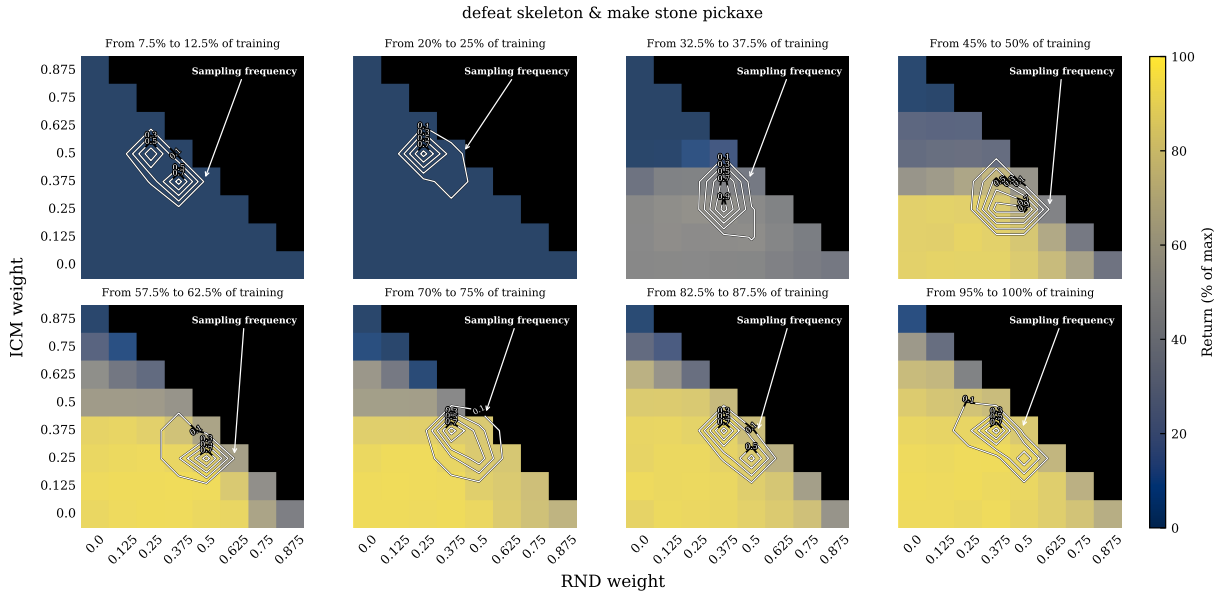


Figure 9: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *defeat skeleton & make stone pickaxe* task, shown at eight intervals comprising 5% of training each. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay the normalized curriculum sampling frequency in weight space over their respective intervals, with labels indicating the contour levels (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

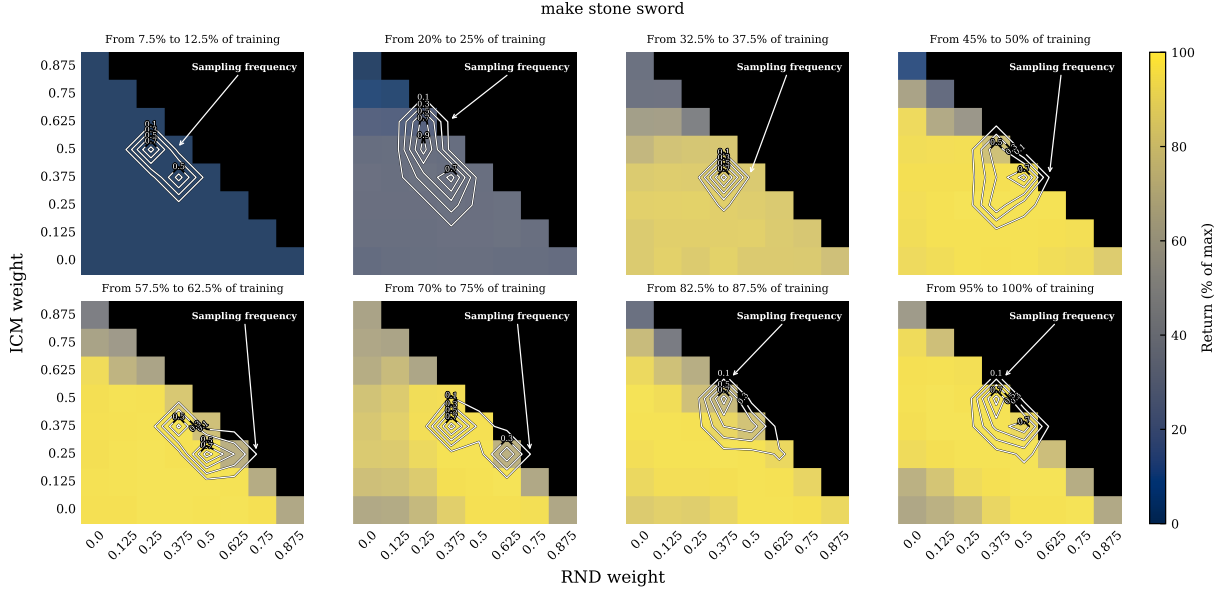


Figure 10: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *make stone sword* task, shown at eight intervals comprising 5% of training each. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay the normalized curriculum sampling frequency in weight space over their respective intervals, with labels indicating the contour levels (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.

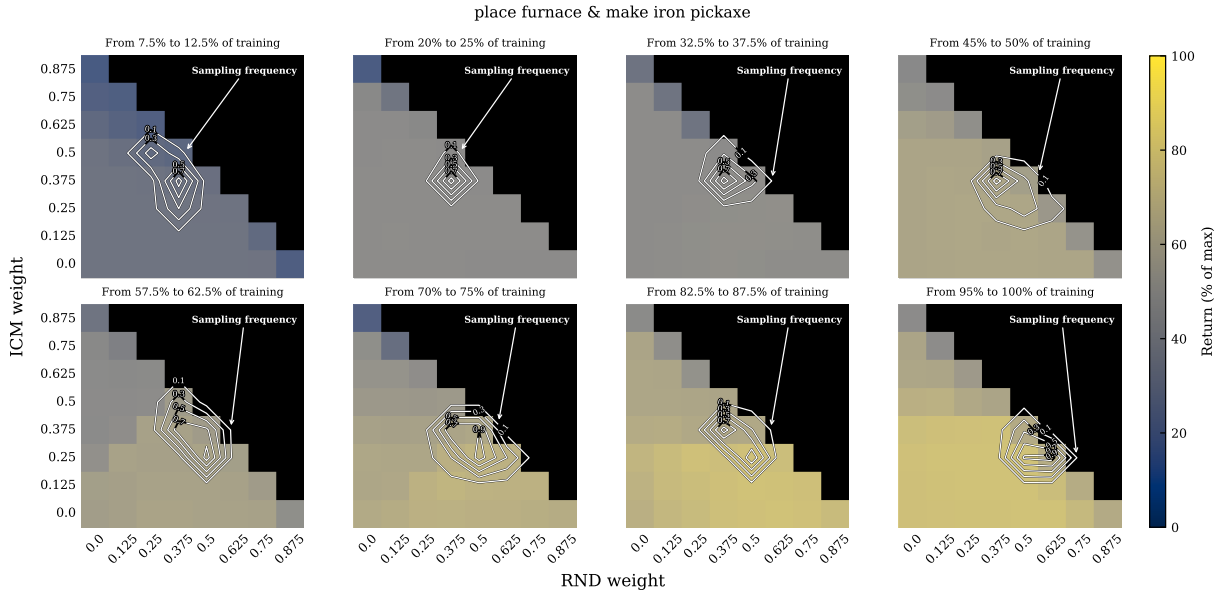


Figure 11: Training-stage snapshots of the CuReMix-conditioned policy landscape for the *place furnace & make iron pickaxe* task, shown at eight intervals comprising 5% of training each. In each panel, the heatmap color encodes the mean extrinsic return across three seeds (as a percentage of the task maximum) for the policy conditioned on each ICM/RND weight pair. Contour lines overlay the normalized curriculum sampling frequency in weight space over their respective intervals, with labels indicating the contour levels (0.1, 0.3, 0.5, 0.7, or 0.9). Black cells mark invalid ICM/RND combinations.